Week 3: Design Smells & Structural Health

Analysis and Assessment of Design Quality

Today's learning goals

1. Separate smells from defects and mere preferences
2. Recognise a core catalog of architecturally significant smells
3. Operationalise coupling/cohesion via change impact
4. Propose checkable mitigations (not rewrite fantasies)

Agenda ($\approx$ 2 hours)

1. Smell vs defect vs taste
2. Architectural significance
3. Core smell catalog
4. Coupling, cohesion, touch count
5. Smells $\leftrightarrow$ attribute risk
6. Mitigations and false alarms

Three different things

Defect	Violates a stated requirement / scenario measure now
Smell	Structure that reliably predicts pain (change cost, incidents, confusion)
Preference	Style without evidenced risk for this brief

Remember

MSc reviews must label which one they are claiming—and show evidence.

Architectural significance

A finding is architecturally significant when it:

- Affects prioritized quality scenarios, or
- Crosses trust / ownership / deployment boundaries, or
- Is expensive to reverse later (irreversibility)

Local naming nits are rarely architecturally significant.

Quiz 1 — Classification

Quiz

Reviewer note: “Package com.acme.util is a dumping ground; payment, auth, and PDF code all import it. Changing invoice PDF layout has broken login tests twice.” Best classification?

1. Preference about package names only
2. Architecturally significant smell (poor cohesion / inappropriate intimacy) with maintainability evidence
3. Security defect proving encryption is broken
4. Non-goal because utils packages are industry standard

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 1

Answer

2. Cross-cutting dumping ground + concrete change breakage = significant smell with evidence.

Core catalog (design / architecture level)

1. God component — owns too many reasons to change
2. Tight / cyclic coupling — changes ripple; cycles freeze evolution
3. Poor cohesion — unrelated responsibilities bundled
4. Leaky layer / abstraction — bypasses defeat the boundary
5. Unclear ownership — who may change this? who on-calls it?
6. Shotgun change — one feature edits many places
7. Ambiguous API contract — unclear errors, versions, idempotency
8. Premature distribution / over-engineering — complexity without scenario demand

God component — signals

- Many inbound dependencies; “centre of the universe”
- Mixes orchestration, rules, persistence, and UI concerns
- On-call nightmares: every incident touches it

Try this

On a dependency sketch, circle the node with highest fan-in. Is it a façade (OK) or a god (risk)?

Premature distribution

Splitting into many services / processes can:

- Raise failure modes (partial outage, dual writes)
- Multiply operational cost
- Fake “modularity” while sharing a database tightly

Common pitfall

Treating “we used microservices” as automatic quality credit.

Quiz 2 — Smell identification

Quiz

Design: eight microservices, one shared database schema, foreign keys across "service" tables, no separate data ownership. Most accurate smell framing?

1. Excellent modularity—eight boxes prove it
2. Premature distribution + hidden coupling via shared DB (distributed monolith risk)
3. Pure usability smell
4. Only a performance optimisation opportunity

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 2

Answer

2. Process boundaries without data ownership yield distributed monolith costs.

Operational definitions

- Coupling — how much one unit knowledge / depends on another
- Cohesion — how strongly responsibilities inside a unit belong together

Practical probe: pick a realistic change; count touch points (modules, APIs, schemas).

Touch-count drill

Feature: “Add gift-card payment.”

1. List expected modules to touch in Design A vs B
2. High touch count $\Rightarrow$ suspect shotgun change / poor boundaries
3. Interpret with scenarios—not absolute thresholds

MSc note

Touch count is a signal. Domain reality sometimes requires multi-module edits.

Quiz 3 — Interpreting coupling

Quiz

Design A: gift-card feature touches 2 modules. Design B: touches 9. Brief prioritises rapid payment-method experiments. Best evaluation comment?

1. B is always better—more modules means more enterprise-ready
2. A is automatically perfect—lowest touch count wins every brief
3. B likely harms the maintainability scenario unless justified; ask for boundary redesign or shared extension point
4. Ignore structure; only UI mockups matter

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 3

Answer

3. Connect metric-like signal to the prioritized scenario; invite justification.

Smells ↔ attribute risk (examples)

Smell	Often threatens
God component	Maintainability, reliability (blast radius)
Leaky layers	Security, maintainability
Ambiguous API	Reliability, interoperability
Unclear ownership	Operability, incident MTTR
Over-distribution	Reliability, cost, modifiability

Checkable mitigations

Prefer small, verifiable moves:

- Extract a module with a named owner and API
- Break a cycle with an interface / event / callback direction
- Move rules to one authority; delete duplicates
- Document trust boundary and enforce in one gateway

Avoid: “rewrite everything in the ideal architecture.”

False alarms

Not every pattern deviation is a smell:

- Prototype / spike with documented expiry
- Performance-critical bypass with tests and comments + non-goal elsewhere
- Small system where extra layers add only cost

Good evaluators argue context.

Quiz 4 — Mitigation quality

Quiz

Finding: cyclic dependency between Orders and Billing. Which mitigation is most checkable for a two-week remediation?

1. "Adopt Clean Architecture everywhere by next sprint"
2. Introduce a one-way domain event OrderPlaced consumed by Billing; forbid Billing→Orders compile-time dependency; add a CI check for the forbidden import
3. Merge both into one god service to remove the cycle
4. Rename packages alphabetically

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — Quiz 4

Answer

2. Directional dependency + automated guardrail is concrete. (1) is fantasy scope; (3) trades one smell for another; (4) is cosmetic.

Key takeaways

- Label smell vs defect vs preference
- Prioritise architecturally significant findings
- Use touch count / dependencies as evidence
- Mitigations must be checkable; watch false alarms

Practice today

Handout: 03-week03-practice.md
Annotate a messy pack; mitigations; one false-alarm debate.

Next week

Design review checklists
Process, roles, severity tags, comment etiquette.